

Асинхронное взаимодействие и брокеры сообщений

Roadmap лекции

01

Зачем говорить о взаимодействии

Обмен данными, сбои, масштабирование

02

От монолита к микросервисам

Плюсы монолита и причины перехода

03

Взаимодействие — архит. задача

Сеть, задержки, таймауты, сбои

04

Синхронное и асинхронное

Сравнение подходов, плюсы и минусы

05

Когда какой подход выбирать

Примеры из домена автосалона

06

Зачем нужен брокер сообщений

Роль в распределённой системе

07

Базовые сущности брокеров

Queue, topic, подписчики

08

Надёжность доставки

ACK, DLQ, at/exactly-once

09

Типовые проблемы message-driven

Дубликаты, порядок, потери сообщений

10

Идемпотентность обработчиков

Повторы и сбои без последствий

Почему эта тема важна

В распределённых системах важно не только разделить систему на сервисы, но и организовать их взаимодействие

- как сервисы обмениваются данными;
- как переживают сбои;
- как масштабируются;
- как реагируют на события

Монолитная архитектура

- всё в одном приложении;
- единая кодовая база;
- прямые вызовы методов;
- один процесс развертывания.

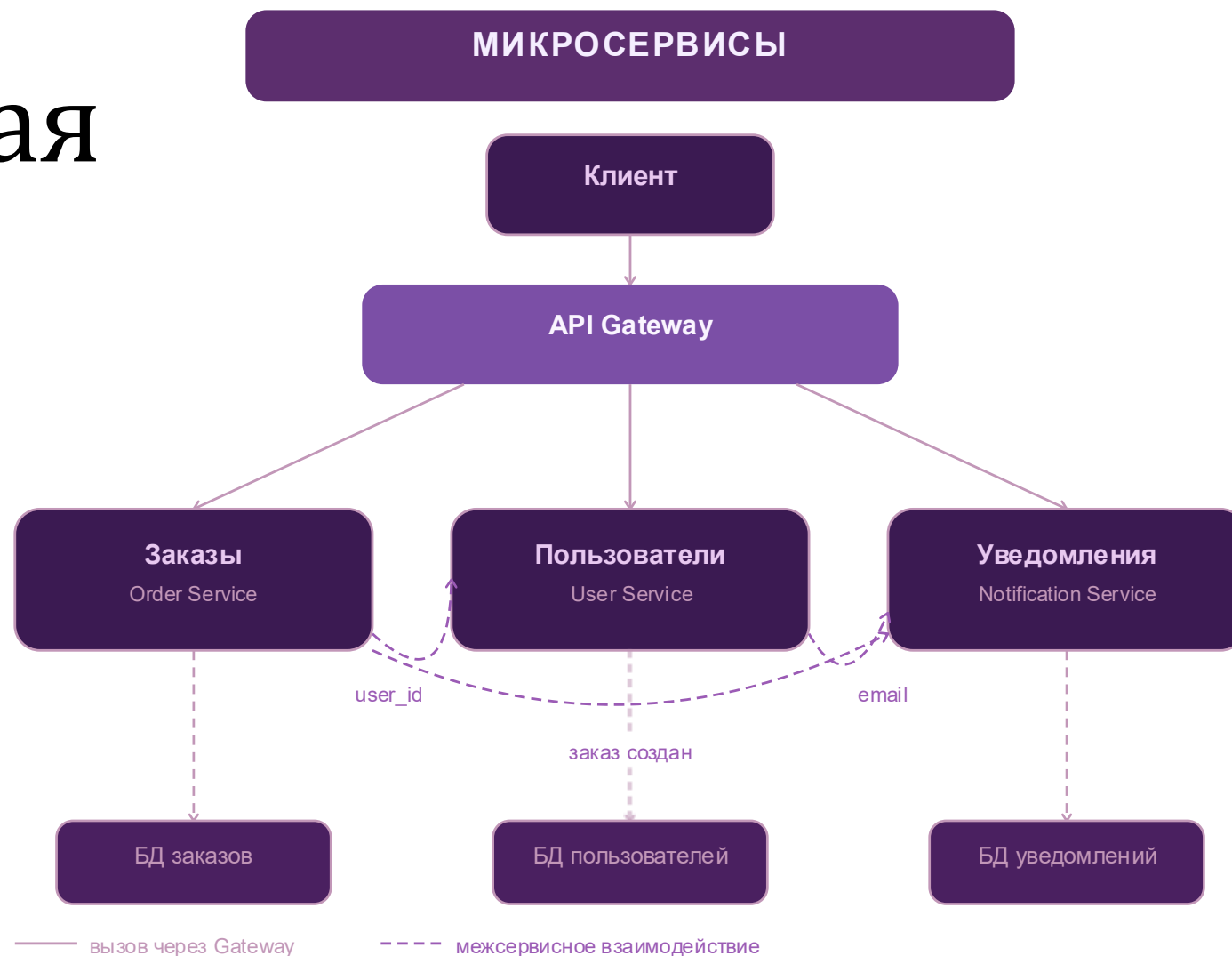
Плюсы

- просто начать;
- просто локально запускать;
- удобно для маленьких систем.



Микросервисная архитектура

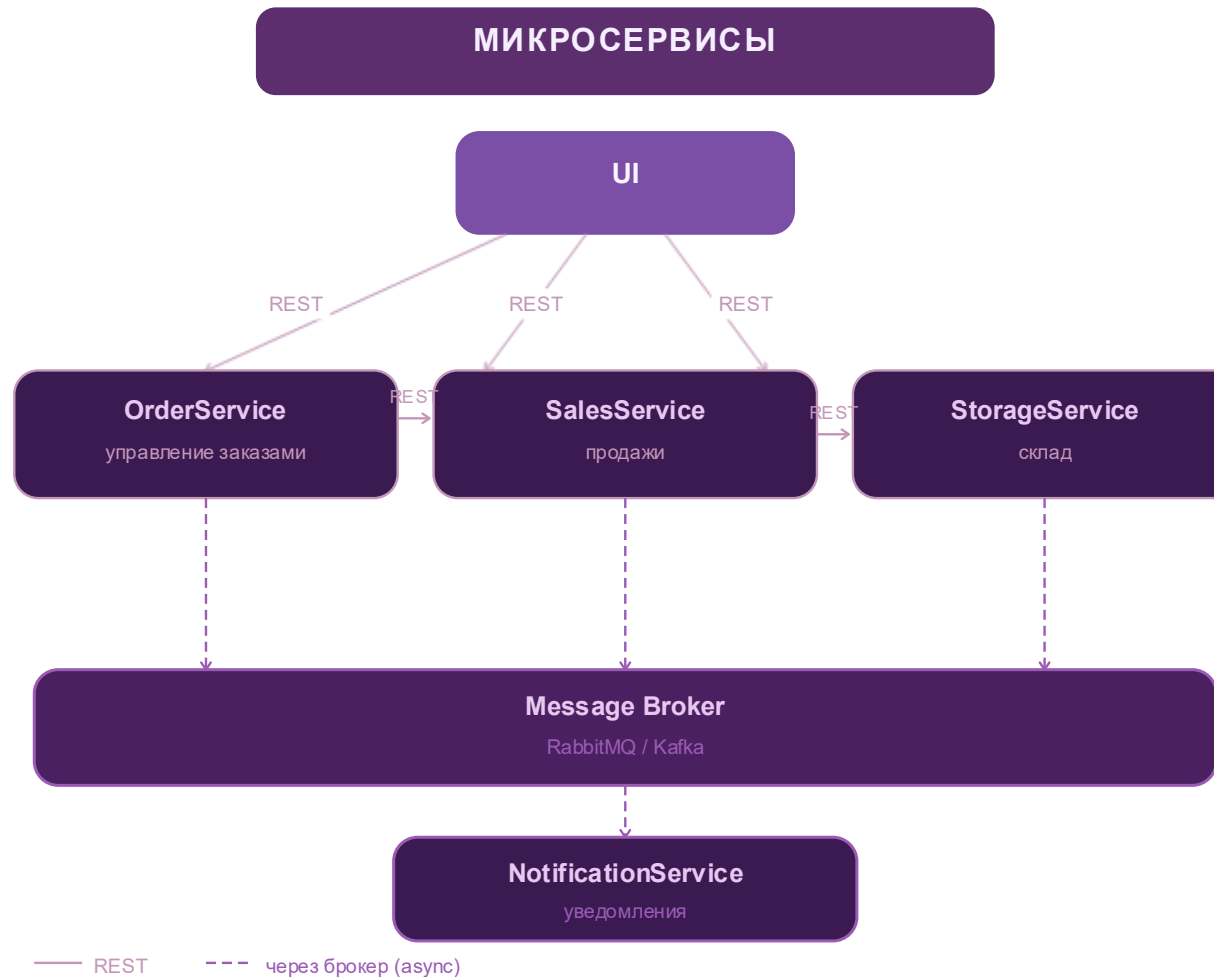
- система состоит из независимых сервисов;
- каждый сервис отвечает за свою бизнес-область;
- у сервиса свой жизненный цикл;
- сервисы взаимодействуют по сети.



Когда МОНОЛИТ становится проблемой?

- рост количества бизнес-сценариев;
- рост команды;
- сложность изменений;
- трудности масштабирования;
- сильная связность модулей;
- долгий цикл поставки изменений;
- хочется использовать технологии, недоступные в текущем стеке

Пример микросервисов автосалона



Что поменялось?

В монолите: вызов метода

В микросервисах: сетевое взаимодействие

Появляются:

- задержки;
- ошибки сети;
- таймауты;
- частичная недоступность;
- необходимость проектировать протокол взаимодействия.

Способы взаимодействия между сервисами

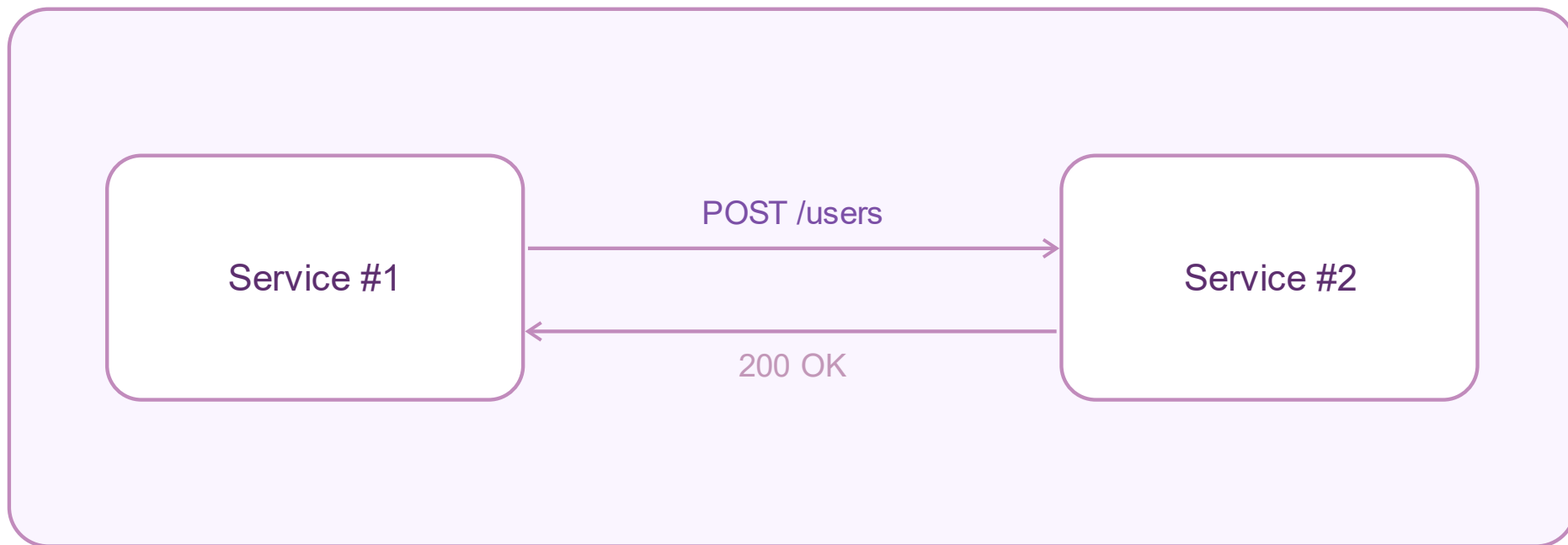
- синхронное взаимодействие;
- асинхронное взаимодействие

Вопросы, которые нужно себе задать:

- нужен ли мгновенный ответ;
- можно ли обработать позже;
- насколько сервисы должны быть связаны;
- как система должна вести себя при сбоях

Синхронное взаимодействие

- Клиент ждёт ответ;
- Блокировка потока;
- Простой, но не масштабируется хорошо;



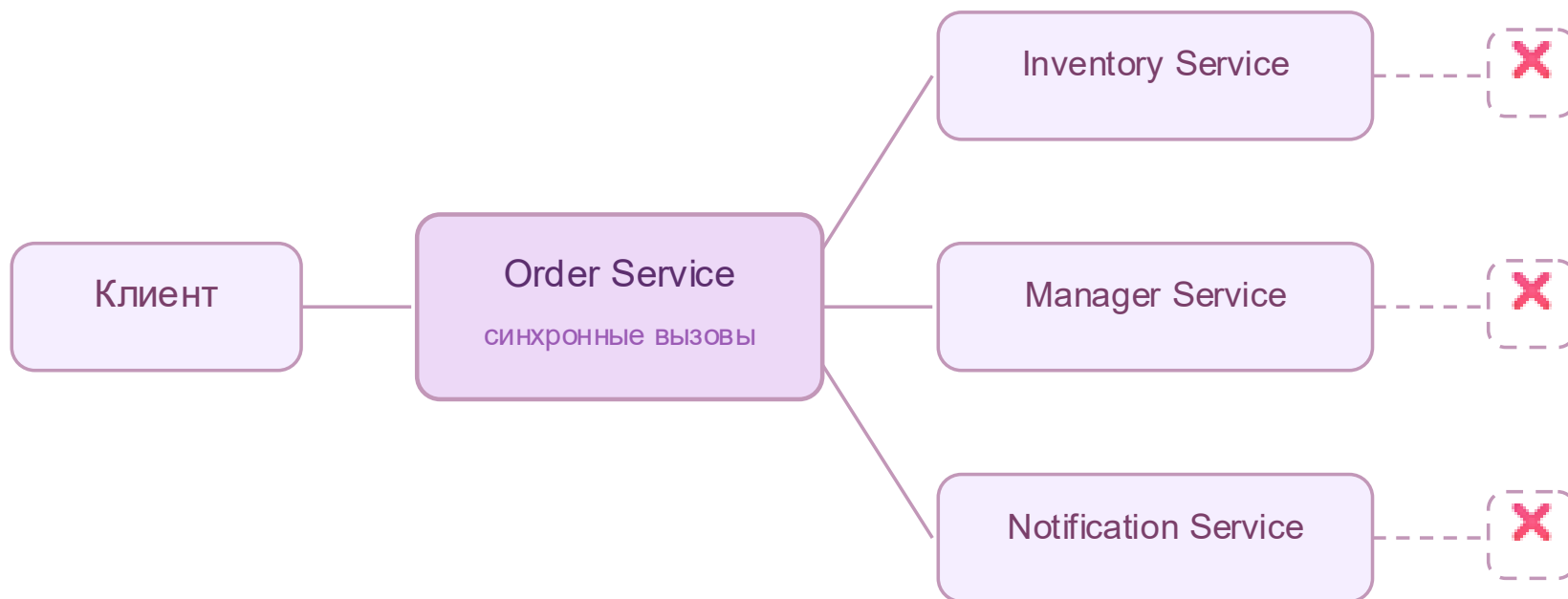
Когда синхронное взаимодействие удобно

- проще понять;
- проще отладить;
- удобно, когда нужен немедленный результат;
- естественная модель для CRUD и чтения данных.

Когда синхронное взаимодействие неудобно

- сильная связность сервисов;
- зависимость по доступности;
- таймауты;
- каскадные сбои;
- ухудшение под нагрузкой

Пример с автосалоном

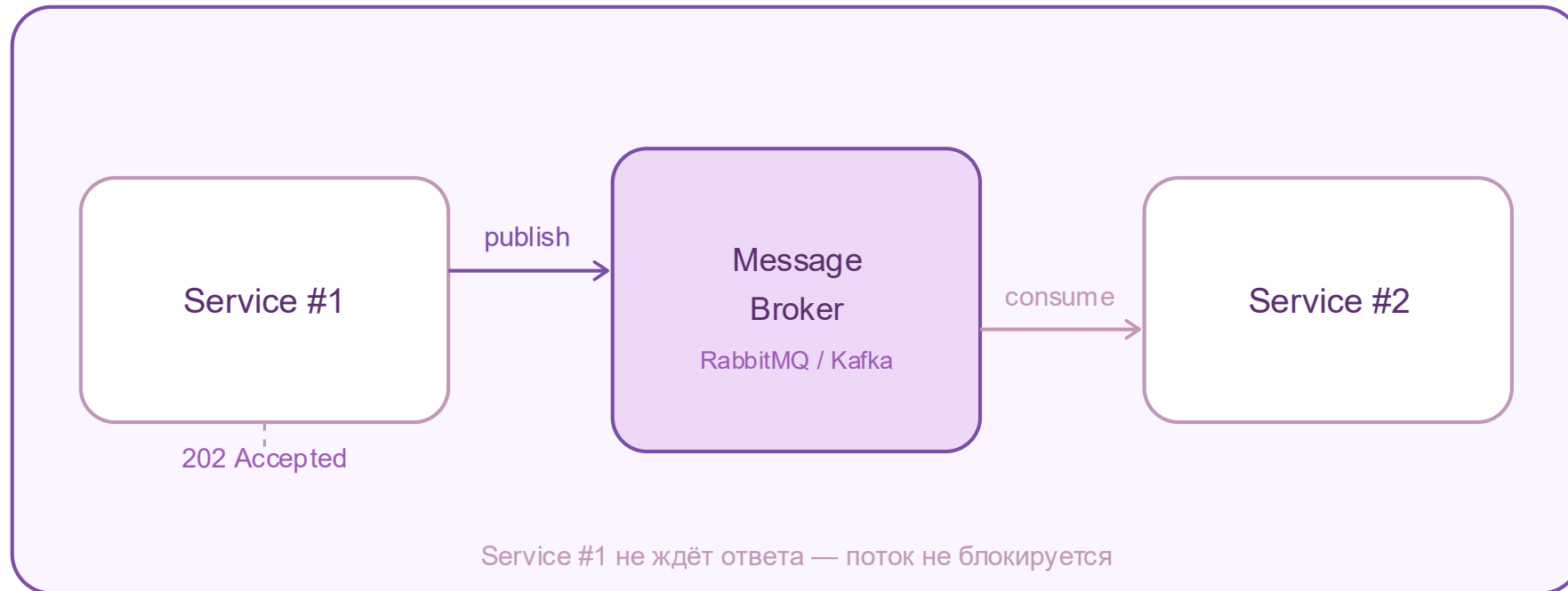


⚠ Риск: синхронная цепочка

Сбой любого сервиса → весь сценарий «Оформление заказа» падает

Асинхронное взаимодействие

- сервис публикует сообщение;
- не ждет немедленного ответа;
- другие сервисы обрабатывают сообщение позже.



Когда асинхронное взаимодействие удобно

- слабая связность;
- выше отказоустойчивость;
- лучше переносит всплески нагрузки;
- удобно подключать новых потребителей;
- легче строить событийную архитектуру.

Когда асинхронное взаимодействие неудобно

- сложнее отладка;
- нет мгновенного ответа;
- возможны дубликаты;
- сложнее трассировать бизнес-процесс

Где синрохонное
где асинрохонное

Просмотр карточки автомобиля

Сценарий:

Клиент открывает страницу конкретного автомобиля в каталоге

Какое взаимодействие между UI и Backend?

Отправка письма после создания заказа

Сценарий:

После оформления заказа клиенту нужно отправить письмо на почту.

Проверка наличия автомобиля перед оформлением

Сценарий:

Перед тем как подтвердить заказ, Order Service должен понять, доступен ли автомобиль на складе.

Оформление заказа на автомобиль в наличии

Сценарий:

Клиент нажимает кнопку “Оформить заказ”.

Нужно:

- создать заказ;
- назначить менеджера;
- зарезервировать автомобиль;
- отправить уведомление клиенту.

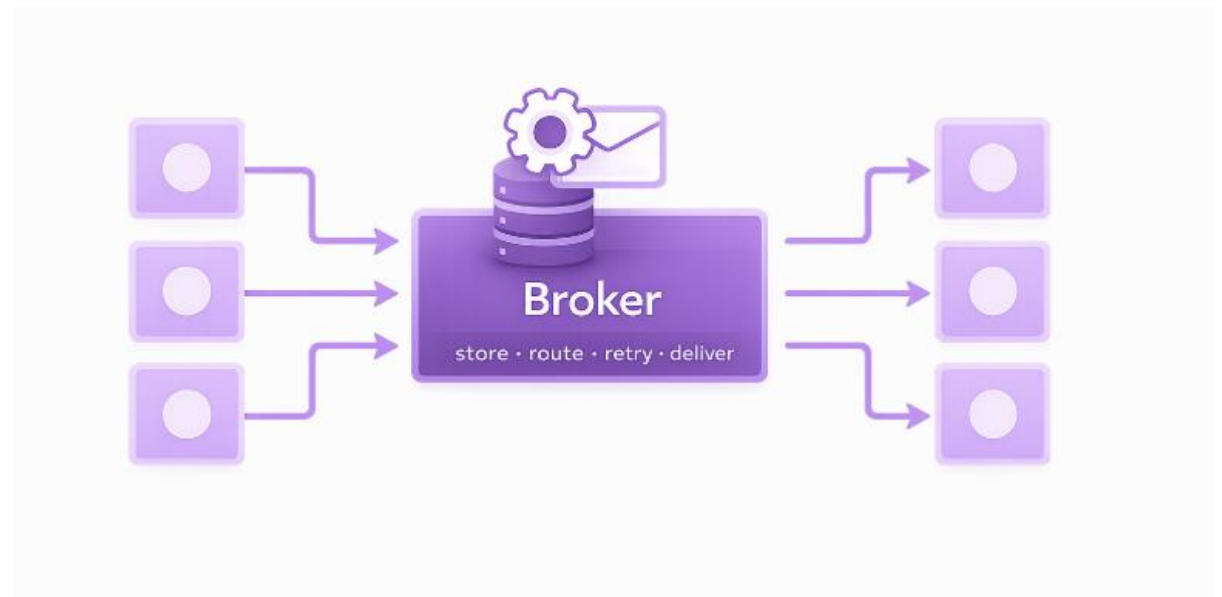


Зачем нужен брокер сообщений

Брокер сообщений – посредник между сервисами

Он выполняет функции:

- приём сообщений;
- хранение сообщений;
- маршрутизация;
- доставка получателям;
- буферизация нагрузки;
- повторная доставка.



Ключевые сущности брокера



Producer (отправитель)

OrderService



Consumer (потребитель)

NotificationService



Message (сообщение)

OrderStatusChanged



Queue или Topic (место доставки)

Queue

сообщение обычно обрабатывает один получатель

подходит, когда:

есть задача;
её должен выполнить кто-то один;
важно распределять нагрузку между обработчиками.

Пример

отправить письмо клиенту;
пересчитать стоимость;
зарезервировать машину;
обработать заявку.

Topic

сообщение может получить несколько подписчиков

подходит, когда:

произошло событие;
об этом должны узнать несколько сервисов;
каждый подписчик решает сам, что делать с этим событием.

Пример

заказ создан;
заказ оплачен;
тест-драйв забронирован.

Какие проблемы возникают

- сообщение пришло дважды;
- сообщение потерялось;
- сообщения пришли не по порядку;
- обработчик упал в середине обработки;
- бизнес-операция выполнялась частично

Подтверждение обработки и повторная доставка

Ack — сообщение обработано

Retry — повторная доставка при ошибке

DLQ — очередь проблемных сообщений

Гарантии доставки сообщений

At most once – максимум один раз

At least once – минимум один раз

Exactly once – очень сложно и дорого

Идемпотентность операции

Идемпотентный обработчик – повторная обработка сообщения не ломает систему

Примеры

- не отправлять один и тот же статус дважды;
- не резервировать один и тот же автомобиль повторно;
- не переводить заказ в уже достигнутый статус повторно.